

DiffFormer: A Diffusion Transformer on Stock Factor Augmentation

Yuan Gao^{§†}, Haokun Chen[‡], Xiang Wang[§], Zhicai Wang[§],
Xue Wang[‡], Jinyang Gao[‡], Bolin Ding[‡]

[§]University of Science and Technology of China, Hefei, China
yuanga@mail.ustc.edu.cn, xiangwang1223@gmail.com, wangzhic@mail.ustc.edu.cn

[‡]DAMO Academy, Alibaba Group, Hangzhou, China
{hankel.chk,xue.w,jinyang.gjy,bolin.ding}@alibaba-inc.com

Abstract—Machine learning models have demonstrated remarkable efficacy and efficiency in a wide range of stock forecasting tasks. However, the inherent challenges of data scarcity, including low signal-to-noise ratio (SNR) and data homogeneity, pose significant obstacles to accurate forecasting. To address this issue, we propose a novel approach that utilizes artificial intelligence-generated samples (AIGS) to enhance the training procedures. In our work, we introduce the **Diffusion Model** to generate stock factors with **Transformer** architecture (**DiffFormer**). **DiffFormer** is initially trained on a large-scale source domain, incorporating conditional guidance so as to capture global joint distribution. When presented with a specific downstream task, we employ **DiffFormer** to augment the training procedure by editing existing samples. This editing step allows us to control the strength of the editing process, determining the extent to which the generated data deviates from the target domain. To evaluate the effectiveness of **DiffFormer** augmented training, we conduct experiments on the CSI300 and CSI800 datasets, employing eight commonly used machine learning models. The proposed method achieves relative improvements of 7.2% and 27.8% in annualized return ratio for the respective datasets. Furthermore, we perform extensive experiments to gain insights into the functionality of **DiffFormer** and its constituent components, elucidating how they address the challenges of data scarcity and enhance the overall model performance.

Our research demonstrates the efficacy of leveraging AIGS and the **DiffFormer** architecture to mitigate data scarcity in stock forecasting tasks. The ability to generate realistic stock factors coupled with the control offered by the editing step presents a promising avenue for improving forecasting accuracy. The experimental results validate the potential of our proposed approach and contribute to a deeper understanding of its underlying mechanisms.

Index Terms—Stock forecasting, Diffusion Model, Transformer, Data Augmentation

I. INTRODUCTION

Accurate stock forecasting plays a crucial role in effective asset management and investment strategies [1]. Its objective is to predict future stock behavior (e.g., return ratios or prices) by analyzing relevant historical factors. Previous research has explored various machine learning techniques, such as SFM [2], ALSTM [3], and HIST [4]. However, achieving desirable performance with these methods often requires an ample supply of high-quality data. The challenges posed by high random and homogeneous data make it difficult to meet the

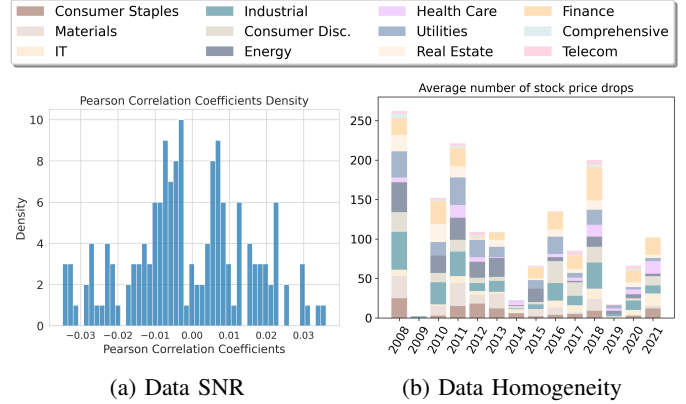


Fig. 1: (a) Pearson Correlation Coefficients between return ratio and stock factors are low. (b) Average number of large price drop stocks split by sectors. Stocks within the same industry sector tend to perform similarly.

requirements for data quality, resulting in elevated forecasting errors and increased uncertainty.

Data scarcity in stock forecasting tasks is commonly characterized from two aspects: *signal-to-noise ratio (SNR)* and *data homogeneity*. (1) We first delve into the relationship between stock factors and the return ratio, providing insights into SNR. Figure 1a visualizes the Pearson Correlation Coefficients between stock factors and the return ratio, revealing a weak correlation (absolute value less than 0.03) indicative of a low SNR of the factors. This weak correlation is often attributed to randomness and non-stationary speculative behaviors. (2) We explore the behavior of stocks within industry sectors to show data homogeneity's implications. We discover that stocks within the same industry sector exhibit similar behavior, as shown in Figure 1b, which reports the average number of large price drop stocks categorized by sectors. As a result, this homogeneity leads to a reduction in the availability of stocks with distinctive informational characteristics. Data scarcity presents inherent challenges that can lead to overfitting, where models exhibit an increased risk of learning shortcuts and spurious correlations, ultimately impacting their predictive performance. The limited availability of data poses a significant hurdle in achieving effective generalization between the training and testing datasets, thereby resulting in compromised

[†] Work done at Alibaba Group.

overall performance.

One straightforward solution to overcome data scarcity is factor augmentation, a technique that involves augmenting the training dataset by introducing minor changes or generating new data points based on specific factors. Drawing inspiration from successful applications of Diffusion Models (DMs) in various domains, such as text-to-image conversion [5], [6], time-series imputation [7], and waveform generation [8], we explore the utilization of DMs for stock forecasting. DMs are generative models consisting of two stages: a diffusion process and a denoising process. The diffusion process parameterizes a Markov chain that progressively introduces noise to the factors until reaching a state of pure noise [9]. Subsequently, during the denoising process, the model aims to restore the original data by predicting the noise generated through the diffusion process. In this study, we look back 8 days and organize recent stock factors as a sequence, leveraging DMs based on transformer architectures [7], [10] to do factor augmentation. We expect that by incorporating augmented factors, our proposed model will exhibit enhanced resilience to data scarcity in the field of stock forecasting.

Incorporating DMs for stock factor augmentation presents non-trivial challenges, particularly in the assignment of corresponding labels to the generated factors. While one possible approach is to directly treat labels as a stock factor dimension, this method carries the risk of generating inaccurate results. Explicitly generating labels presents challenges due to the difficulty of accurately matching factors with their corresponding labels, particularly considering the unpredictability of return ratios. To overcome this challenge, we propose to adapt DMs from generation task to supervised-learning task by utilization of a flexible predictor-free conditional factor generator [5], [10], [11]. The generator is trained with labels as the conditioning, allowing us to anticipate that the generated factors will share the same labels as the original factors. Additionally, we explore the utilization of other types of conditionings (*i.e.*, sectors) to further enhance accuracy. An illustration is shown in Figure 2.

To distill new knowledge and information (*i.e.*, stocks in other markets) during augmentation, our proposed framework incorporates transfer learning. Initially, the DM is trained in a large source domain with a diffusion step denoted as T ; during the inference process, instead of generating from a standard normal distribution, we start from data points in the target domain, corrupt them and then denoise to obtain a new data point that is likely in the target domain. However, due to the requirement of a large T for DMs to reach a state of pure noise, the low SNR prevalent in stock data poses challenges in achieving accurate recovery as T increases. In practical implementation, we limit the corruption of data to a smaller number of steps denoted as $T' \ll T$, which we refer to as the editing step. In addition, we have discovered a significant benefit resulting from this mechanism. The phenomenon known as data collision occurs when multiple models utilize the same copy of data simultaneously. In such cases, a surge of capital flowing into the market can lead to the failure of these

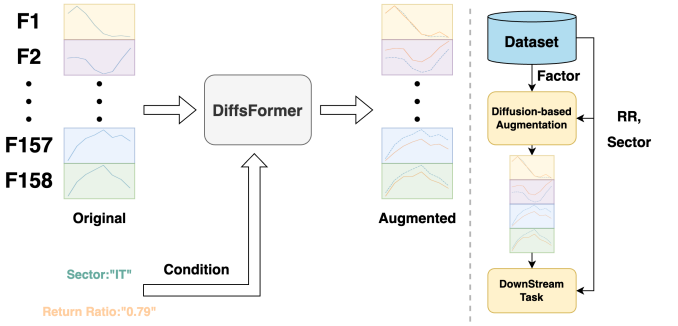


Fig. 2: An illustration of DiffFormer. F refers to “factors”, such as the open, close, lowest, and highest prices of a stock.

models in achieving their investment objectives [12]. However, through the editing of factors, this issue can be effectively mitigated by introducing new copies of data. To summarize, our approach involves editing samples in the target domain using knowledge derived from a larger source domain.

We also propose several improvements to increase the model’s efficiency and decrease the volatility. It is obvious that there is no need to optimize the DM for $t > T'$. On top of that, we initialize the DM with T to ensure its correctness, but sample training steps t from $\text{Uniform}\{1, 2, \dots, T'\}$ instead of $\text{Uniform}\{1, 2, \dots, T\}$. Additionally, we leverage the training loss as a proxy and introduce stronger noise to data points with lower training loss. This loss-guided noise addition mechanism aims help decrease model volatility by mitigating the overfitting issue associated with easily fitted points, in contrast to uniform noise addition.

In summary, the contributions are as follows:

- We reveal the importance of data augmentation in the context of stock forecasting and explore the use of **diffusion stock transformer** (DiffFormer for short) to address data scarcity.
- To adapt DM from generative task to supervised-learning task, we propose to employ ground truth (*e.g.*, return ratio) as the conditioning to enhance the relationship between factors and label. Furthermore, we enhance the flexibility of the guidance by integrating a predictor-free guidance approach.
- Due to the low SNR of factors, we propose to edit the existing samples in a transfer learning manner instead of synthesizing new ones. One advantage of this mechanism lies in its capacity to distill new knowledge in contrast to methods like random noise augmentation.
- We verify the effectiveness of DiffFormer augmented training in CSI300 and CSI800 with eight commonly used machine learning models. The proposed method achieves 7.2% and 27.8% relative improvements in annualized return ratio respectively.

II. RELATED WORKS

In this section, we will introduce the related work in stock forecasting and discuss some time-series diffusion models.

A. Stock Forecasting

Stock forecasting is a field that utilizes historical time-series data to predict future stock prices. Machine learning models, particularly time-series models such as LSTM, GRU, and Bi-LSTM, have gained popularity in this domain [1], [13]–[15].

Researchers have proposed tailored models to better fit the financial scenario. For example, Li et al. [16] introduce extra input gates to extract positive and negative correlations between factors. Ding et al. [17] propose a novel LSTM model to simultaneously predict the opening, lowest, and highest prices of a stock. Agarwal et al. [18] propose a hybrid prediction model (HPM) that combines three time-series models. Zhang et al. [2] propose a State Frequency Memory (SFM) network that decomposes the hidden states of memory cells into multiple frequency components to model different latent trading patterns. Feng et al. [3] incorporate a temporal attentive aggregation layer and adversarial training into an LSTM variant. Chen et al. [19] use Bi-LSTM to encode stock data and financial news representations in their Structured Stock Prediction Model (SSPM) and multi-task Structured Stock Prediction Model (MSSPM).

CNNs are also believed to capture important features for predicting stock fluctuations. For instance, Deng et al. [20] propose the Knowledge-Driven Temporal Convolutional Network (KDTCN), which integrates knowledge graphs with CNNs to fully utilize industrial relations. Lu et al. [21] enhance a CNN-based model by extracting historical influential stock fluctuations with attention mechanism. Chandar [22] transforms technical indicators into images and used them as input for a CNN model.

To handle non-Euclidean structured data, some researchers have incorporated Graph Neural Networks (GNNs) into stock forecasting. Velickovic et al. [23] construct a graph with stocks as nodes and used graph attention network (GAT) to aggregate neighbor embeddings. Xu et al. [24] construct a stock market relationship graph and extracted information hierarchically. Li et al. [25] propose an LSTM Relational Graph Convolutional Network (LSTM-RGCN) model that handles both positive and negative correlations among stocks.

The Transformer model [26], with self-attention and positional encoding mechanisms, has shown great potential in stock forecasting. Ding et al. [27] improve the Transformer by incorporating multi-scale Gaussian prior, optimizing locality, and implementing Orthogonal Regularization. Yoo et al. [28] propose a Data-axis Transformer with Multi-Level Contexts (DTML) to learn the correlations between stocks. Yang et al. [29] introduce the Hierarchical, Transformer-based, multi-task (HTML) model for predicting short-term and long-term asset volatility. For more related works, please refer to [1].

B. Time-series Diffusion Models

Diffusion models (DMs) are a family of generative models based on deep learning. DMs have demonstrated impressive performance in generating data samples that adhere to the observed data distribution, thus making them a powerful

tool for data augmentation. In recent years, DMs have been implemented for multivariate time series (MVTs) applications.

The first prominent model in this domain is TimeGrad, proposed by Rasul et al. [30]. TimeGrad builds upon Denoising Diffusion Probabilistic Models (DDPM) and consists of a forward process and a reverse process that are conditioned on an RNN module, which encodes historical information. Yan et al. [31] extend TimeGrad from discrete to continuous by constructing the diffusion process using stochastic differential equations (SDEs). They also discuss the potential of extending RNN to other architectural structures.

More recently, Li et al. [32] introduce D3VAE to address noisy time series. They designed a novel coupled diffusion process to synthesize features and labels, respectively, and employed a bidirectional variational auto-encoder (BVAE) to bridge the two.

Spatio-temporal graphs (STG) represent a unique form of time-series data that encodes both spatial and temporal interactions among nodes in a graph [33]. Wen et al. [33] introduce DiffSTG, which applies DMs on STG by incorporating a graph convolution layer into the traditional UNet architecture. The model has a DDPM-like structure that is conditioned on both the graph and the temporal information.

For more related works, please refer to [34].

III. BACKGROUND

In this section, we will introduce some definitions in our work and the problem of stock price forecasting.

A. Problem Formulation

Definition 1. Stock Factors. Factors are attributes of a stock that are identified as potential drivers of return.

Remark. To provide clarity, stock factors can be classified into two types: Alpha factors and Beta factors. Alpha factors measure excess returns, while Beta factors measure volatility. Representative examples of these factors include stock volume, value (*i.e.*, open, high, close, low prices), and momentum.

Definition 2. Return Ratio (RR). The primary objective of stock forecasting is to achieve substantial profits. Previous study [1] treat RR as a metric to measure the model performance. RR serves as a crucial indicator to assess the success of stock forecasting models in achieving profitable investment outcomes. Following this setting, we define return ratio as:

$$RR(i) = \frac{Cprice_{t+i} - Cprice_t}{Cprice_t}, \quad (1)$$

where t is the current time, and i denotes the time interval in days. $Cprice_t$ denotes the current close price of the stock, and $Cprice_{t+i}$ represents the close price of the same stock after i days. Here, we calculate the return ratio on a daily basis, and often set i to be 5.

Definition 3. Stock Forecasting. Based on specific predefined stock factors, our goal is to train a time-series regressor that can accurately predict the future stock behavior of the test stocks in the next few days.

Algorithm 1 DiffsFormer Training

Input: stock data $\mathbf{X} \in \mathbb{R}^{n \times d \times k}$, diffusion step T
for $t = 1$ to T **do**
 initialize β_t and calculate $\bar{\alpha}_t$
end for
Select an editing step $T' \leq T$
while Not Converge **do**
 $i \sim \text{Uniform}\{1, 2, \dots, n\}$
 $t \sim \text{Uniform}\{1, 2, \dots, T'\}$
 $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
 $\mathbf{x}_0 := \mathbf{X}[i]$
 calculate $\mathbf{x}_t$ given $\mathbf{x}_0$ with Eq.(4)
 calculate $\mathcal{L}_{da}$ with Eq.(6)
 Take a gradient step on $\nabla_{\theta} \mathcal{L}_{da}$
end while

B. Denoising Diffusion Probabilistic Model

Denoising Diffusion Probabilistic Models (DDPM) have achieved impressive performance in various domains, especially in text-to-image scenarios [5], [6]. Typically, training a DM needs diffusion and denoising processes.

Diffusion process. Given a data point $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, the diffusion process gradually adds noise to construct a sequence of step-dependent variables $\{\mathbf{x}_t\}_{t=1}^T$ [35]. More specifically, it forms a Markov chain as [7]:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad (2)$$

where $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$. $\mathcal{N}$ denotes the Gaussian distribution, α_t controls the strength of signal retention, and β_t controls the scale of the added noise. These two scalars are predefined for each step t , and one commonly used setting is the variance preserving process [9] where $\alpha_t = 1 - \beta_t$.

Denoising process. The goal of the denoising process is to reconstruct the corresponding noise vector by inverting the transformations performed in the diffusion process. This process is defined by another Markov chain [7]:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad (3)$$

where $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$. p_{θ} is the distribution estimation of q , for which $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_{\theta}(\mathbf{x}_t, t)\mathbf{I})$. Concretely, for each sample in the batch, a time step t is uniformly sampled from $\{1, 2, \dots, T\}$, followed by the adjustment of the noise at time t .

Inference process. Once θ is well-trained, the DM can generate samples from the standard Gaussian distribution with $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ and then repeatedly recover $\mathbf{x}_T \rightarrow \dots \rightarrow \mathbf{x}_t \rightarrow \mathbf{x}_{t-1} \rightarrow \dots \rightarrow \mathbf{x}_0$ given $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$. As $T \rightarrow \infty$, the generative process modeled with Gaussian conditional distributions becomes a good approximation.

Algorithm 2 DiffsFormer Inference

Input: number of data to be generated m , sampling steps l , conditionings c (if guidance is enabled), editing step T' selected during training
while Generated point $< m$ **do**
 $i \sim \text{Uniform}\{1, 2, \dots, n\}$
 $\mathbf{x}_0 := \mathbf{X}[i]$
 calculate $\mathbf{x}_{T'}$ given $\mathbf{x}_0$ with Eq.(4)
 $\mathbf{x}_{\tau_l} := \mathbf{x}_{T'}$
 for $t = l$ to 0 **do**
 calculate $\mathbf{x}_{\tau_{t-1}}$ with DDIM sampling
 end for
end while

IV. METHODOLOGY

The stock forecasting task is challenging primarily because of the scarcity of data. To harness the full potential of machine learning models, a sufficient amount of high-quality data is crucial. However, obtaining such high-quality stock data for a specific target domain is rare and can often be restricted as commercial secrets. In this work, we utilize the power of DM and introduce a novel approach, DiffsFormer. It generates additional data points and facilitates factor augmentation, enabling us to forecast the likely RR of real-world stocks despite data scarcity.

A. Diffusion-based Data Augmentation

Following [5], [9], DiffsFormer contains diffusion and denoising processes like most of the DMs do.

Diffusion process. In stock forecasting, the input data $\mathbf{X} \in \mathbb{R}^{n \times d \times k}$ consists of n real stocks along with their recent k -day historical factors, for which d is the factor dimension. We treat each stock $\mathbf{x}$ (i.e., a row of $\mathbf{X}$) as $\mathbf{x}_0$ sampled from $q(\mathbf{x}_0)$, and add random noise to perform a transition according to Eq.(2). Thanks to the reparameterization trick [9], we can obtain the conditional distribution $q(\mathbf{x}_t|\mathbf{x}_0)$ for each stock [7], [35]:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}), \quad (4)$$

where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ and $\alpha_t = 1 - \beta_t$. Then, $\mathbf{x}_t$ is approximated as $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + (\sqrt{1 - \bar{\alpha}_t})\epsilon$ where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.

Denoising process. During the denoising process, we subtract noise from $\mathbf{x}_t$ to recover the corresponding $\hat{\mathbf{x}}_0 \sim q(\mathbf{x}_0)$. As mentioned in §III-B, we parameterize $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ through a neural network to estimate $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$. Specifically, we have $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_q(t)\mathbf{I})$ with:

$$\begin{aligned} \mu_{\theta}(\mathbf{x}_t, t) &= \frac{1}{\sqrt{\alpha_t}}(\mathbf{x} - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_{\theta}(\mathbf{x}_t, t)) \\ \Sigma_q(t) &= \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t}, \end{aligned} \quad (5)$$

where $\epsilon_{\theta}(\mathbf{x}_t, t)$ is the trainable noise term to predict ϵ in the diffusion process.

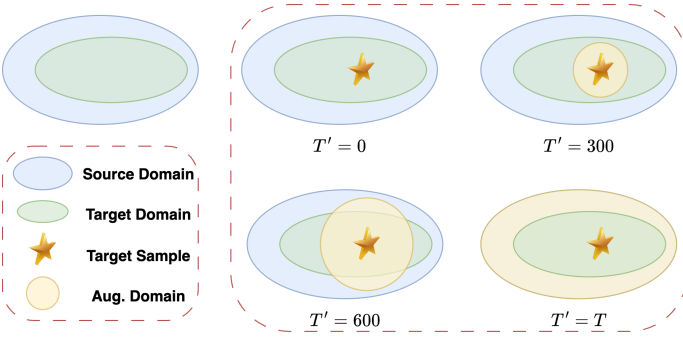


Fig. 3: Illustration of the editing step.

Objective. The overall learning objective is to minimize the error in estimating ϵ using $\epsilon_\theta(\mathbf{x}_t, t)$ [5]. Formally, we aim to solve the following optimization problem:

$$\mathcal{L}_{da} = \min_{\theta} \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(0, \mathbf{I}), t \sim \text{Uniform}} \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2 \quad (6)$$

$$\text{s.t. } \mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + (\sqrt{1 - \alpha_t}) \epsilon.$$

Inference acceleration. In DDPM, the lack of parallelism during the transition of DMs leads to slow inference. To tackle this problem, DDIM [36] accelerates samplings by modifying the forward process as a non-Markov process, i.e.,

$$q_\sigma(\mathbf{x}_{1:T} | \mathbf{x}_0) = q_\sigma(\mathbf{x}_T | \mathbf{x}_0) \prod_{t=2}^T q_\sigma(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0), \quad (7)$$

where $q_\sigma(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ is parameterized by σ which means the magnitude of the stochastic process. When setting $\sigma_t = \Sigma_q(t)$ for all steps, the forward process collapses to Markovian and the denoising process becomes the same as shown in Eq.(5). Specifically, when setting $\sigma_t = 0$, the corresponding denoising process becomes deterministic and thus sampling could be accelerated along the deterministic path. Technically, we follow the deterministic sampling design and create $\{\tau_i\}, \{i = 1 \dots l\}$ as a sub-sequence of $\{t = 1, 2, \dots, T\}$, where l is the length of the sub-sequence. The denoising process can now be completed in just $l \ll T$ steps armed with DDIM sampling.

Factor editing with transfer learning. To alleviate data homogeneity issue, we augment the raw factors in target domain by going through a noising-denoising process. Instead of generating synthetic factors from pure noise which hardly ensures data fidelity, we adopt a different approach by editing the original factors rather than generating entirely new ones. Moreover, due to the intrinsic low SNR nature of the factors, we design a transfer learning framework to distill new knowledge and information into edited data from a larger, different domain. Concretely, DiffFormer of diffusion step T is first trained on the source domain $\mathbf{X}^{(s)}$. During the inference process, we begin with a data point in the target domain $\mathbf{x}_0^{(t)}$, corrupt it for $T' \ll T$ steps to get a seed point: $\mathbf{x}_0^{(t)} \rightarrow \mathbf{x}_1^{(t)} \rightarrow \dots \rightarrow \mathbf{x}_{T'}^{(t)}$. Then, we reverse the process from the seed to obtain a new data point $\hat{\mathbf{x}}_{T'}^{(t)}$ in the target domain: $\mathbf{x}_{T'}^{(t)} \rightarrow \mathbf{x}_{T'-1}^{(t)} \rightarrow \dots \rightarrow \hat{\mathbf{x}}_0^{(t)}$. Since the

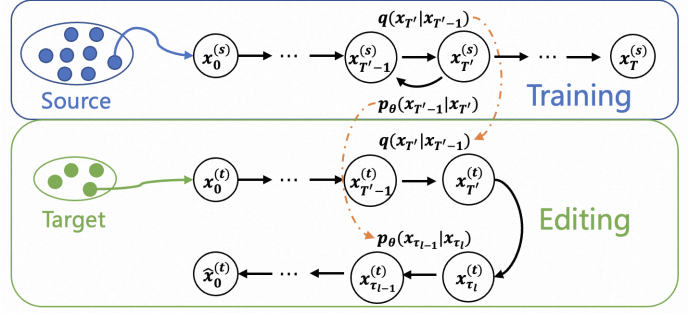


Fig. 4: The training and the editing topology.

target domain is a subset of the source domain, this procedure distills new knowledge and information and enhances the data heterogeneity. Moreover, since the inference process starts from the seed, we can successfully edit existing samples. As illustrated in Figure 3, T' can control the strength of knowledge distillation: a larger T' makes the generated data resemble the feature distribution of the source domain more closely, while a smaller T' makes the generated data closer to the target domain data $\mathbf{x}_0^{(t)}$. We term T' as the editing step. By doing so, we improve the fidelity of the generated data, avoiding creating data from pure noise. Recent works [37] term this trick as “real guidance”. An illustration of the process is shown in Figure 4.

Time efficiency improvements. In our transfer learning framework, it is obvious that there is no need to optimize $\epsilon_\theta(\mathbf{x}_t, t)$ for $t > T'$. On top of that, we develop a trick to speed up the training of the framework. Concretely, we initialize α and β with the total diffusion steps T ; but during training, we sample training steps t from $\text{Uniform}\{1, 2, \dots, T'\}$ instead of $\text{Uniform}\{1, 2, \dots, T\}$. We discover that this modification results in a sharper loss curve during training as shown in Figure 13. The detailed algorithms for training and inference are shown in Algorithms 1 and 2, respectively.

B. Conditional Diffusion Augmentation

Most generative tasks do not have the demands for label generation. However, in the stock forecasting task, a clean and informative supervised signal is essential for training the regressor. Directly appending the label to the factor vector is ineffective. Taking Figure 5 as an example, we calculate the R^2 score between the augmented and the original factors (label). It is observed that the label has the least correlation among the 159 dimensions. Furthermore, we report an experimental comparison between the settings of label-generation and label-conditioning in Table I. We suppose that generated label fails to serve as the accurate supervised signal for the generated feature. As an alternative, we pave the way to control the synthesis process through guidance inputs, including labels and industry information [38]. At the meantime, we keep the labels of the generated features the same as those of the original features to adapt DMs from the generative task to the regression task.

TABLE I: Model performance with label-generation and label-condition mechanisms

	Label-generation	Label-condition
Model Performance	0.159327	0.312679

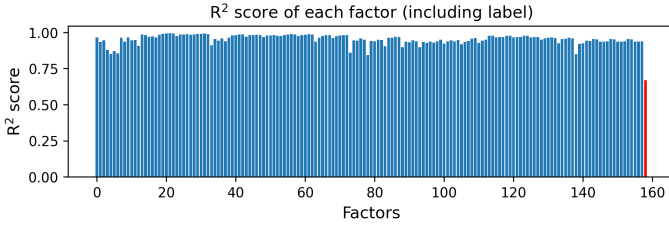


Fig. 5: The R^2 score between the generated and the original factors and label. R^2 score is the square of the Pearson Correlation. The blue bars represent the R^2 scores of 158 factors, while the red bar shows the R^2 score of the label.

1) *Predictor Guidance*: One straightforward approach is to introduce an auxiliary classifier ϕ that estimates $p_\phi(y|\mathbf{x}_t, t)$ [39], [40]. The gradients $\nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t, t)$ obtained during training guide the synthesis of data points in the inference process. Concretely, $\epsilon_\theta(\mathbf{x}_t, t)$ in Eq.(6) becomes:

$$\bar{\epsilon}_\theta(\mathbf{x}_t, t, y) = \epsilon_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t \omega} \nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t, t), \quad (8)$$

where ω is a scalar to control the guidance strength.

This guiding format is widely used to introduce new concepts in image synthesis [39], [40]. However, in our work, the target label is continuous, which is largely unexplored to the best of our knowledge. One of the differences between continuous and discrete label space is the use of time step t . Note that in discrete space, a noisy image may belong to an unknown class, especially when it is pure noise. Therefore, the classifier takes time step t as input to incorporate diffusion information. In contrast, in our work, every sequence is meaningful since the expected label is continuous. On top of that, we refine Eq.(8) by decoupling the training of the predictor from DMs and ignoring time step t in the input:

$$\bar{\epsilon}_\theta(\mathbf{x}_t, t, y) = \epsilon_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t \omega} \nabla_{\mathbf{x}_t} (p_{\phi^*}(\mathbf{x}_t) - y)^2, \quad (9)$$

where ϕ^* is the pre-trained separate predictor whose parameters remain fixed during the inference. Similar to other modules, we choose the Transformer as the auxiliary predictor for its excellent performance in sequence tasks. One of the advantages of the predictor guidance mechanism is that it maintains the plug-and-play design of the framework, making our model scalable.

2) *Predictor-free Guidance*: Besides predictor guidance, an alternate approach is the predictor-free guidance. Due to the fact that the additional predictor complicates the training pipeline, some research work explore to eliminate it [5], [10], [11]. Moreover, fusing the obtained gradient into DM training can be considered a gradient-based adversarial attack [11], making the performance of predictor guidance poorer than that of the predictor-free guidance in most cases [5], [10].

Therefore, in our work, we also explore the implementation of the predictor-free guidance mechanism.

Upon closer inspection of Eq.(9), we observe that predictor guidance works by changing the sampling distribution from $p_\theta(\mathbf{x}_t|y)$ to $\bar{p}_\theta(\mathbf{x}_t|y)$ where:

$$\bar{p}_\theta(\mathbf{x}_t|y) \propto p_\theta(\mathbf{x}_t|y) p_\theta(y|\mathbf{x}_t)^\omega. \quad (10)$$

As the guidance strength ω gets larger, DM receives more rewards when generating $\mathbf{x}_t$ for which the auxiliary predictor assigns a high probability $p_\theta(y|\mathbf{x}_t)$ to the ground truth y . According to [11], the same effect can be achieved by jointly training conditional and unconditional DMs. Specifically, the inference process is performed in the form of:

$$\hat{\epsilon}_\theta(\mathbf{x}_t, c) = \epsilon_\theta(\mathbf{x}_t, \emptyset) + \omega_{free} \cdot (\epsilon_\theta(\mathbf{x}_t, c) - \epsilon_\theta(\mathbf{x}_t, \emptyset)), \quad (11)$$

where c denotes the condition vectors and $\emptyset$ denotes a learnable null vector. During training, c is randomly replaced with $\emptyset$ with a fixed probability to train an unconditional DM. Note that ω_{free} shall be greater than 1 in predictor-free guidance. The advantages of predictor-free guidance are: 1) it is a simple approach since no auxiliary predictor is needed; 2) it is flexible since it supports other types of conditionings beyond return-ratio labels. In our work, we further explore the use of industry information. We observe that stocks in different industries tend to perform in different patterns. For instance, financial stocks (e.g., banks) usually have low yields but enjoy low volatility, while many information technology stocks have high yields but undertake high volatility. Furthermore, we can synthesize industry-specific data to improve model performance in specific industry sector. One of the unappealing properties of the predictor-free guidance is that it injects the conditionings during the training of DMs. As a result, when adding or modifying conditionings, we need to retrain the DMs although it is time-consuming.

C. Model Details

1) *Model Overview*: Figure 6 elucidates the overall framework of our stock price forecasting model. Generally, it consists of a conditioned transformer-based data augmentation module and a backbone return ratio predictor. The framework is designed with several considerations: 1) DMs acts as a plug-and-play data augmentation module, so it can be deployed to different backbones without retraining; 2) our data is organized in sequences, so we explore the use of transformers to better capture the autocorrelation in the sequence, as opposed to the commonly adopted UNet [41] in text-to-image generation models; 3) the transfer-based editing framework distills new knowledge and information, meanwhile prevents the new data copy from deviating from the original data too much.

2) *Architecture Improvement*: Following [10], we adjust the transformer structure to fit predictor-free guidance. Specifically, as shown in Figure 6, we feed the diffusion steps into the transformer module. Furthermore, it contains an adaptive layer norm module and a zero-initialized scalar module.

Time step encoding. During the training, DiffsFormer needs to know which diffusion step it is trained for. In our work,

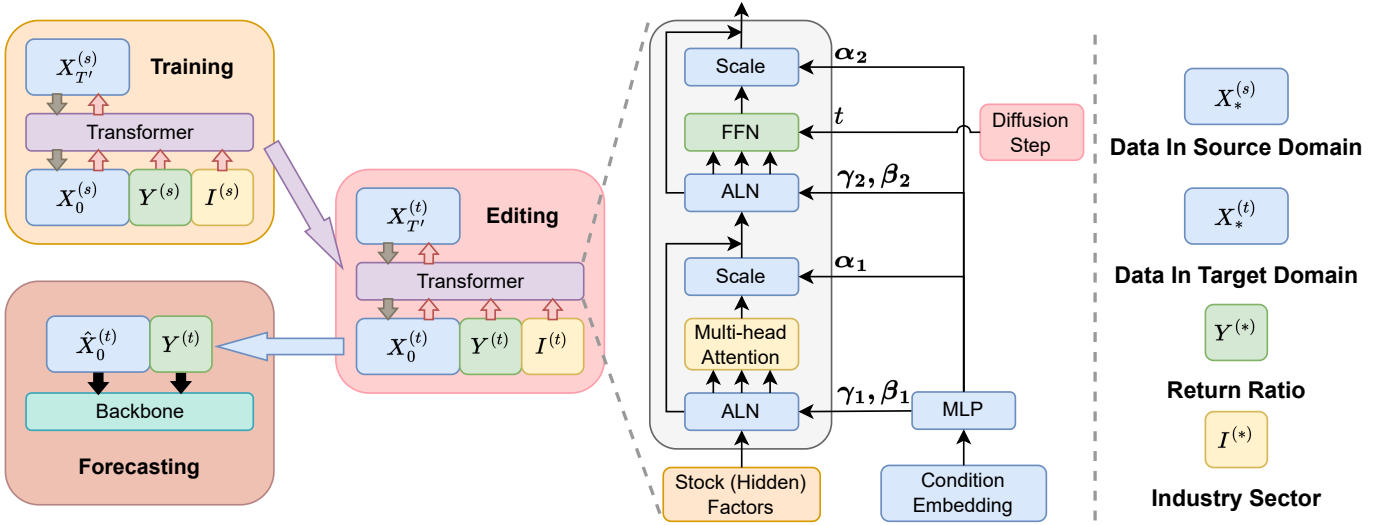


Fig. 6: DiffsFormer overview. Y denotes label and I denotes industry sector. DiffsFormer is initially trained on a large-scale source domain, incorporating conditional guidance to ensure improved model performance. When presented with a specific downstream task, we employ DiffsFormer to augment the training procedure by editing existing samples.

we encode the current diffusion step into *Sinusoidal positional encoding* and add it to the feed forward network. This embedding scheme is appropriate to encode the time step information, the positional encoding at position $p + k$ can be linearly represented by the positional encoding at position p .

Adaptive layer norm (ALN). Adaptive layer norm [42] are widely adopted in generative models [40], [43]. In the transformer block, we append an additional ALN layer after the standard layer norm. Notably, ALN regresses the scale and shift parameters γ and β from two affine transformations f and h to the sum of conditioning vectors.

Zero initialization. In addition to ALN layer, many DMs [9], [10], [40] also incorporate zero initialization in the framework, meaning that the model parameters are initialized as zero such that the conditioning is ineffective when training just starts. In this case, MLP is initialized to make α_1 and α_2 equal to 0, and thus transformer block becomes an identity function [10], [44] (i.e., input tokens are directly fed to the next layer).

V. EXPERIMENT

In this section, we conduct experiments on the real-world stock data from 2008 to 2022 provided by [45] with the aim of answering the following research questions:

- **RQ1:** Is proposed DiffsFormer flexible to various backbones and helpful in enhancing them?
- **RQ2:** Can DiffsFormer alleviate data scarcity? How effective are the proposed components?
- **RQ3:** How does DiffsFormer help control the volatility of the backbone models?

A. Dataset

1) *Dataset:* Following [4], [46], we evaluate the proposed framework on two real-world stock datasets: CSI 300 and CSI 800. The CSI 300 comprise the largest 300 stocks traded on the

TABLE II: Hyper-parameters and the search range, the optimal parameters are indicated in boldface.

Parameters	Search Range
editing step during inference	{200, 300, 400 , 500}
layers in DM	{3, 6 }
stop loss thred	{0.6, 0.8, 0.9, 0.95, 0.965 , 1}
batch norm	{ False , True}
norm first	{False, True }
guidance strength	{1.1, 2, 3 , 4}
industry condition	{False, True }
label condition	{False, True }
market value condition	{ False , True}
market value groups	{3, 10 }

Shanghai Stock Exchange and the Shenzhen Stock Exchange¹, and represents the performance of the whole A-share market in China. CSI 800 is a larger dataset consisting of CSI 500 and CSI 300, aiming to add some stocks with smaller size. Note that DiffsFormer aims at editing the existing samples with new information from a larger domain. Hence in practice, we use all stocks in the China A-share market to train the DM and editing on CSI 300 and CSI 800, respectively.

2) *Factors:* We use the Alpha158 factors provided by the AI-oriented quantitative investment platform Qlib². These factors review the basic stock information including *kbar*, *price*, *volume*, and *some rolling factors* in different time windows. For each stock at date t , we look back 8 days to construct a sequence of factor as $\mathbf{x} \in \mathbb{R}^{8 \times 158}$. During the time span between 2008-01-01 and 2022-09-30, the number of sequences is 2109804. Hence our input matrix $\mathbf{X}$ is of shape $2109804 \times 8 \times 158$.

¹https://en.wikipedia.org/wiki/CSI_300_Index

²<https://github.com/microsoft/qlib>

B. Reproducibility

In this subsection, we introduce some details of the proposed work for easier reproduction.

1) *Robust Z-score Normalization*: Generally, the values between factors are not in the same scale. To address this issue, we adopt *Robust Z-score Normalization* within stocks. Based on z-score, robust z-score replace mean and standard deviation with median (MED) and the median absolute deviation (MAD). In robust statistical methods³, MED is the robust measure of central tendency, while mean is not; MAD is robust measure of statistical dispersion, while standard deviation is not. Specifically, the i th input stock data is normalized to:

$$\hat{x}[i] = |x[i] - \text{MED}(\mathbf{X})| / \text{MAD}(\mathbf{X}). \quad (12)$$

2) *Dropping Extreme Label*: When a stock hits limit up(limit down), stockholders are more reluctant to sell (buy) at this time, for they expect that the trend continues; as a result, it is difficult for other stockholders to buy (sell). Therefore, it is meaningless for the model to learn to “buy when there is a limit up, and sell when there is a limit down”. To tackle this challenge, we propose to drop the extreme label to exclude the influence of extreme values. It is achieved in two ways: 1) we set a upper threshold and a lower threshold; 2) we drop the first and the last few percent labels.

3) *Software and Hardware*: DiffsFormer is implemented with Python 3.8.16, Pytorch 1.11.0. All of the backbones are implemented with the open-sourced code in Qlib. We run the experiments on servers equipped with NVIDIA Tesla V100 GPU and 2.50GHz Intel Xeon Platinum 8163 CPU.

4) *Model Parameters*: In Table II, we summarize the main hyper-parameters of DiffsFormer, as well as their search ranges. The optimal hyper-parameter are highlighted.

C. Experimental Setup

1) *Baselines*: To verify the performance of the proposed framework in stock forecasting, we employ eight commonly used machine learning models as forecasting backbones:

- **MLP**: a 2-layer multi-layer perceptron (MLP) with the number of units on each layer is 256.
- **LSTM** [13]: a Long Short-Term Memory network based stock price forecasting method.
- **GRU** [14]: a Gated Recurrent Unit (GRU) network based stock price forecasting method.
- **SFM** [2]: a State Frequency Memory (SFM) network that decomposes the hidden states of memory cells into multiple frequency components to model different latent trading patterns.
- **GAT** [23]: a graph is constructed with stocks as nodes, and two stocks have a relation when they share the same stock concept. Then graph attention network (GAT) is utilized to aggregate the neighbor embedding attentively.
- **ALSTM** [3]: an LSTM variant that incorporates temporal attentive aggregation layer to aggregate information from hidden embeddings in previous timestamps.

- **Transformer** [26]: a transformer [26] based stock price forecasting model.
- **HIST** [4]: a graph-based framework that mines the concept-oriented shared information from predefined concepts and hidden concepts. The proposed framework simultaneously utilize the stock’s shared information and individual information.

2) *Evaluation Metrics*: Since our task is stock forecasting, we utilize **Annualized Return Ratio (RR)** as the primary evaluation metric. In practice, we simulate the stock trade with “top30drop30” strategy: “top30” means that we keep the stocks with top30 predicted scores; and “drop30” means that each stock will be dropped if its score falls out of top30, regardless of its previous performance. Besides, we also use two widely used metrics: the **Information Coefficient (IC)** [47] and **Ranked Information Coefficient (Rank IC)** [48]. IC denotes the Pearson correlation between prediction and label, while Rank IC denotes the Spearman’s rank correlation between prediction and label. To eliminate the effect of random seed and other cause of performance fluctuation, we run the training and testing procedure 8 times for all of the methods and report the average value and the standard deviation. Since the training of DMs and the predictor is decoupled, we only run DM once for time efficiency.

D. Performance Comparison

To answer **RQ1**, we perform a completed comparison between the original and the augmented feature on the mentioned baselines, wherein the percentage of relative improvement on each metric is shown in Table III and IV. Note that HIST requires the concept of stocks to build the graph, therefore we don’t run it on CSI800 where the concepts are not available. Another notion is that the test time range is 2017-01-01 to 2020-12-31 in previous works [4], [46], which is not consistent with **2020-04-01** to **2022-09-30** in our work. The reason is that we find factors and model performance can decays with age, and we aim to provide with an up-to-date performance of the models. As a result, the performance of backbones in our paper and that in previous works are not comparable. The main observation are as follows:

- In general, the proposed framework DiffsFormer improve the performance of backbone models by 0.50% ~ 13.19% and 4.01% ~ 70.84% on return ratio on CSI 300 and CSI 800, respectively. This observation empirically verifies the necessity of data augmentation strategy in stock forecasting.
- Return ratio is the primary performance metric since the ultimate goal of stock forecasting is to achieve substantial profits. We observe that IC and RankIC are not positively associated to return ratio. We attribute the cause to our dataset CSI300 and CSI800 where going short is banned. Hence accurate modeling of tail stocks has little contribution to return ratio compared to that of top stocks. In alternative, we introduce to apply an exponentially decayed weight on IC to better characterize the correlation between the prediction and label on top stocks.

³https://en.wikipedia.org/wiki/Robust_statistics

TABLE III: Performance comparison on CSI300. The better results are indicated in boldface.

Methods	CSI300								
	RR			IC			RankIC		
	original	augmentation	improvements	original	augmentation	improvements	original	augmentation	improvements
MLP	0.2093 $\pm$ 0.0300	0.2163$\pm$0.0210	3.34%	0.0508 $\pm$ 0.0044	0.0537$\pm$0.0026	5.71%	0.0499 $\pm$ 0.0059	0.0509$\pm$0.0034	2.00%
LSTM	0.2312 $\pm$ 0.0308	0.2336$\pm$0.0219	1.04%	0.0516$\pm$0.0022	0.0429 $\pm$ 0.0026	-16.86%	0.0519$\pm$0.0021	0.0455 $\pm$ 0.0021	-12.33%
GRU	0.2161 $\pm$ 0.0293	0.2413$\pm$0.0149	11.66%	0.0536$\pm$0.0038	0.0511 $\pm$ 0.0012	-4.66%	0.0552$\pm$0.0037	0.0516 $\pm$ 0.0012	-6.52%
SFM	0.2189 $\pm$ 0.0325	0.2200$\pm$0.0175	0.50%	0.0505 $\pm$ 0.0018	0.0510$\pm$0.0025	0.99%	0.0507 $\pm$ 0.0026	0.0526$\pm$0.0029	3.75%
GAT	0.2461 $\pm$ 0.0176	0.2701$\pm$0.0168	9.75%	0.0558$\pm$0.0012	0.0532 $\pm$ 0.0007	-4.66%	0.0540 $\pm$ 0.0014	0.0551$\pm$0.0006	2.04%
ALSTM	0.2047 $\pm$ 0.0351	0.2317$\pm$0.0233	13.19%	0.0502$\pm$0.0027	0.0450 $\pm$ 0.0023	-10.36%	0.0510$\pm$0.0031	0.0439 $\pm$ 0.0019	-13.92%
HIST	0.2272 $\pm$ 0.0352	0.2410$\pm$0.0207	6.07%	0.0547$\pm$0.0011	0.0518 $\pm$ 0.0032	-5.30%	0.0545$\pm$0.0023	0.0535 $\pm$ 0.0025	-1.83%
Transformer	0.2789 $\pm$ 0.0376	0.3127$\pm$0.0113	12.12%	0.0598 $\pm$ 0.0031	0.0603$\pm$0.0025	0.83%	0.0638 $\pm$ 0.0024	0.0672$\pm$0.0017	5.33%

TABLE IV: Performance comparison on CSI800. The better results are indicated in boldface.

Methods	CSI800								
	RR			IC			RankIC		
	original	augmentation	improvements	original	augmentation	improvements	original	augmentation	improvements
MLP	0.1037 $\pm$ 0.0383	0.1161$\pm$0.0223	11.96%	0.0386 $\pm$ 0.0023	0.0399$\pm$0.0006	3.37%	0.0450 $\pm$ 0.0048	0.0467$\pm$0.0035	3.78%
LSTM	0.1248 $\pm$ 0.0282	0.1298$\pm$0.0317	4.01%	0.0377 $\pm$ 0.0017	0.0412$\pm$0.0008	9.28%	0.0500$\pm$0.0030	0.0494 $\pm$ 0.0010	-1.20%
GRU	0.0758 $\pm$ 0.0307	0.1295$\pm$0.0292	70.84%	0.0380$\pm$0.0026	0.0376 $\pm$ 0.0010	-1.05%	0.0493 $\pm$ 0.0030	0.0511$\pm$0.0011	3.65%
SFM	0.0906 $\pm$ 0.0413	0.1250$\pm$0.0375	37.97%	0.0385$\pm$0.0005	0.0365 $\pm$ 0.0015	-5.19%	0.0485 $\pm$ 0.0011	0.0487$\pm$0.0022	0.41%
GAT	0.1814 $\pm$ 0.0309	0.2013$\pm$0.0210	10.97%	0.0379 $\pm$ 0.0005	0.0397$\pm$0.0003	4.75%	0.0483 $\pm$ 0.0009	0.0483 $\pm$ 0.0009	0.00%
ALSTM	0.1030 $\pm$ 0.0253	0.1518$\pm$0.0290	50.29%	0.0316 $\pm$ 0.0031	0.0383$\pm$0.0013	21.20%	0.0418 $\pm$ 0.0034	0.0492$\pm$0.0015	17.70%
Transformer	0.1751 $\pm$ 0.0386	0.1903$\pm$0.0382	8.68%	0.0423 $\pm$ 0.0028	0.0426$\pm$0.0018	0.71%	0.0573$\pm$0.0016	0.0556 $\pm$ 0.0022	-2.97%

TABLE V: Weighted IC Comparison on CSI800 and CSI300.

Methods	CSI300		CSI800	
	original	augmentation	original	augmentation
MLP	0.0326 $\pm$ 0.0023	0.0332$\pm$0.0021	0.0052 $\pm$ 0.0041	0.0063$\pm$0.0032
LSTM	0.0295 $\pm$ 0.0032	0.0339$\pm$0.0025	0.0075$\pm$0.0055	0.0024 $\pm$ 0.0026
GRU	0.0324 $\pm$ 0.0012	0.0383$\pm$0.0011	0.0005 $\pm$ 0.0027	0.0128$\pm$0.0029
SFM	0.0288 $\pm$ 0.0029	0.0300$\pm$0.0030	0.0028$\pm$0.0032	0.0026 $\pm$ 0.0030
GAT	0.0354$\pm$0.0006	0.0324 $\pm$ 0.0004	0.0083$\pm$0.0010	0.0047 $\pm$ 0.0008
ALSTM	0.0260 $\pm$ 0.0038	0.0312$\pm$0.0033	0.0025 $\pm$ 0.0064	0.0094$\pm$0.0023
HIST	0.0249 $\pm$ 0.0066	0.0317$\pm$0.0026	-	-
Transformer	0.0387 $\pm$ 0.0038	0.0433$\pm$0.0048	0.0066 $\pm$ 0.0058	0.0159$\pm$0.0054

The weighted result is reported in Table V. From the table, we can observe that: 1) weighted IC for CSI800 is obviously lower than that for CSI300, which is consistent with return ratio performance in Table III and IV. 2) The models' rankings in terms of weighted IC and return ratio are similar, especially on CSI800, suggesting weighted IC can be served as a metric to measure the potential of reaching a high return ratio. 3) DiffsFormer boosts the weighted IC for most of the methods on the CSI300 and improves the weighted IC for more than half of the methods on the CSI800, verifying its effectiveness of improving model performance.

- Due to the low SNR of data, we surprisingly observe that the performance of backbone methods with original features are counterintuitive (e.g., ALSTM, as well as HIST which is one of the best methods in stock forecasting, are inferior to LSTM). Data augmentation to some extent gets

the performance level back to normal. This observation suggests that data augmentation could alleviate the low SNR issue of financial data.

E. Discussion

To answer RQ2, we elaborate on the quality of the edited data, and investigate the effectiveness of our proposed novel components.

1) *Data Fidelity and Diversity*: Our work adopts diffusion-based data augmentation module to synthesize data, which helps alleviate the serious data scarcity issue in stock forecasting. Particularly, before the start of training of the predictor in each epoch, we generate a new set of stock data. Therefore, the total amount of data utilized is $n \times$ the original where n is equal to the total number of epoches. In other words, with DiffsFormer, the backbone can observe $n \times$ the data for once, instead of observing original data for n times. To measure the fidelity and diversity of the data generated, two metrics are commonly adopted [9], [10], [38]: Fréchet Inception Distance (FID) and Inception Score (IS). FID measures the distance between means and standards of two Gaussian distributions, given that two Gaussian distributions are close if their mean and standards are similar, hence a smaller FID indicates high fidelity; IS measures the KL Divergence between $p(y|x)$ and $p(y)$, given that $p(y|x)$ is sharp if data has high quality and $p(y)$ is flat if data has great diversity. However, $p(y|x)$ requires a well-trained classifier (e.g., Inception Network), instead we use model performance to measure both the diversity and the fidelity of the data. As reported in Figure 8a, FID decreases as the guidance strength gets stronger, and DiffsFormer could

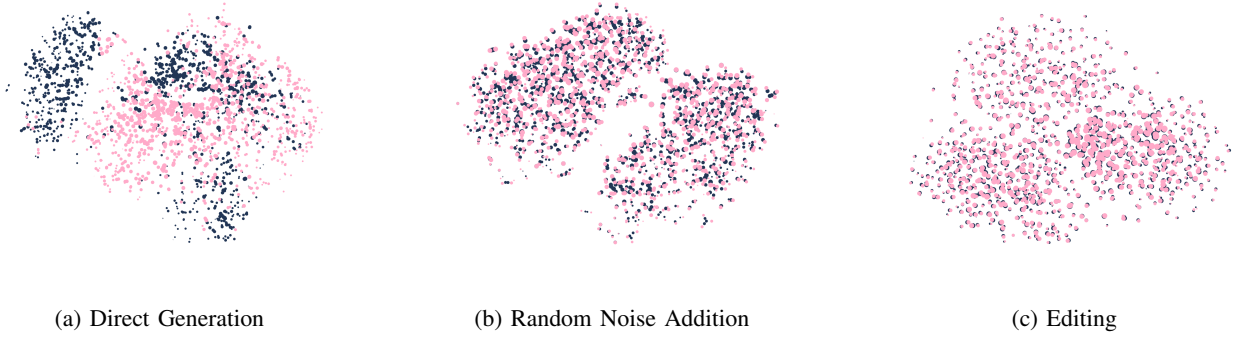


Fig. 7: t-SNE plots of original features (pink) and augmented features (blue) elucidating the effect of editing, (b) denotes random gaussian addition. The lack of separation in the feature space suggests that the edited factors are meaningful.

TABLE VI: The Effect of Editing Steps

Editing Step	200	300	400	500
Performance	0.284258	0.312679	0.293627	0.271165
FID	0.411302	0.690823	1.138044	1.892653

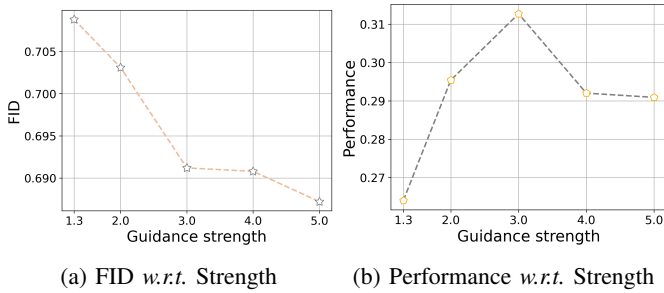


Fig. 8: Illustration of Data Fidelity and Diversity

achieve FID as low as 0.6872, suggesting a high fidelity of the generated data. This finding also verifies the effectiveness of editing mechanism, as DiffsFormer could achieve 0.7088 FID. In addition, from Figure 8b, we find that strong guidance strength may lead to performance drop. We attribute the reason for the phenomenon to the lack of diversity of the data.

2) *Editing vs. Generating*: Due to the low SNR of the data, we train DM in the source domain that is much larger than target domain. During inference, we start from data point in the target domain, and corrupt them for a few steps before reversing to obtain a new data point in the target domain. In Figure 7, following recent work [49], we visualize the relationship between the augmented features and the original stock features in blue and pink, respectively. We have two observations: 1) Comparing Figure 7a and Figure 7c, we find generated data are restricted to locate near the original data when we edit the existing sample from the target domain; while many points deviate the target domain distribution when we directly synthesize new data points. 2) Random gaussian noise addition can be treated as a special augmentation mechanism. We run several experiments with different level random gaussian noise addition and plot in Figure 7b the t-SNE of feature distribution

TABLE VII: Data augmentation and fine-tuning results on different target and source domains.

Mechanism		Fine Tuning	Diffusion DA
Target Domain	Source Domain	Return Ratio	
CSI800	CSI800	0.1751 $\pm$ 0.0386	0.1793 $\pm$ 0.0113
	CSIS	0.1641 $\pm$ 0.0300	0.1903 $\pm$ 0.0382
CSI300	CSI300	0.2789 $\pm$ 0.0376	0.2861 $\pm$ 0.0547
	CSI800	0.2773 $\pm$ 0.0181	0.2789 $\pm$ 0.0333
	CSIS	0.2432 $\pm$ 0.0372	0.3127 $\pm$ 0.0113

TABLE VIII: The Stock Forecasting Performance with Different Conditionings

		Performance	FID
Original Unconditioned Predictor Guidance		0.278941	-
		0.291862	0.900919
		0.302396	0.900562
Predictor-free Guidance	Label	0.297099	0.733522
	Industry	0.300917	0.822593
	Label+Industry	0.312679	0.690823

with the most accurate return ratio prediction. Our proposed method looks better than random noise addition.

Since the target domain is a subset of the source domain, we distill new knowledge and information and enhance the data heterogeneity. Editing step T' during inference can control the strength of knowledge distillation: a larger T' makes generated data resemble more feature distribution from the source domain, while a smaller T' makes edited data closer to original target domain data. To support this argument, we report the editing steps along with corresponding model performance and FID between the original and the edited data in Table VI. We observe a trade-off between model performance and the editing step, which we attribute to the increased data diversity in the very early diffusion steps and the decreased data fidelity in the later steps.

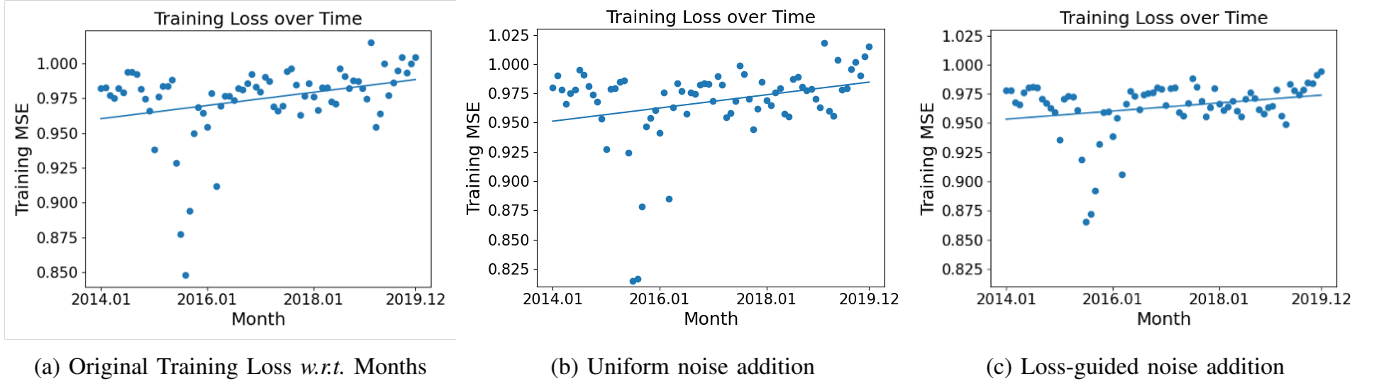


Fig. 9: The illustration of the impact of loss-guided diffusion.

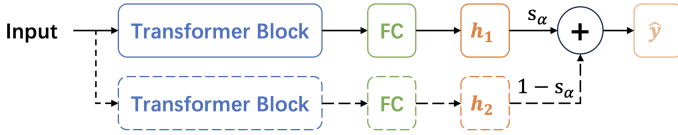


Fig. 10: Illustration of Shake-shake with Transformer.

F. Effectiveness Analysis

1) *Transfer Diffusion*: Recall that in §IV-A, we design a novel inference process to distill new knowledge to generated data through transfer learning. To verify the real cause of performance improvement, we aim to exclude the interference of the new information. The result is shown in Table VII, where the fine tuning column denotes the mechanism of training on source domain and testing on target domain, and diffusion DA stands for diffusion-based data augmentation with transfer learning. Our observation are three-folds: 1) Training in a larger source domain and testing in the target domain may degrade the model performance although new information is introduced, we suppose the reason is the distribution difference between the source domain and the target domain. 2) When the source domain and the target domain are the same, which means no new information is involved, the DM still can help raise the performance. 3) The transfer diffusion boosts the model performance by a large margin, demonstrating the effectiveness of the transfer learning mechanism.

2) *Conditional Diffusion*: To adapt DM from generative task to regression task, we adopt predictor guidance and predictor-free guidance approaches. The stock forecasting performance with different conditionings are reported in Table VIII. We observe that DMs achieve lower FID and contribute to a better model performance with the help of conditionings.

3) *Comparison with Other Augmentation Algorithms*: In this work, we reveal that data augmentation plays a pivotal role in stock forecasting. And in this section, we aim to verify the DiffsFormer’s superiority over other data augmentation mechanisms. The experimental results are reported in Figure 11. The box from left to right are: 1) origin feature; 2) random gaussian noise addition; 3) shake-shake augmentation; 4) DiffsFormer. Shake-shake regularization [50] is first proposed to help alleviate the overfitting problem in deep

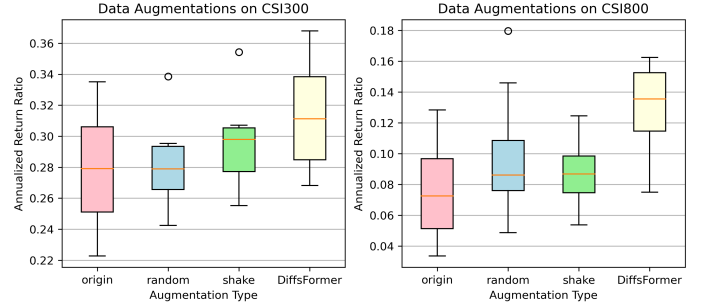


Fig. 11: Comparison between different augmentation methods on CSI300 with Transformer and CSI800 with GRU.

learning. The core idea is to replace the standard forward layer with a stochastic affine combination of parallel branches; to make it clearer, a toy illustration example is shown in Figure 10. In each epoch, we sample s_α stochastically and *keep* it instead of resampling s_β for backward propagation. From this perspective, shake-keep could be treated as doubling the original data. From Figure 11, we observe that: 1) Directly tweaking feature with noise addition can bring performance improvement, indicating that data collision is severe in dataset and the SNR of the data is quite low. 2) Shake-shake and DiffsFormer are two effective data augmentation mechanisms that outperform the random gaussian noise addition, and our proposed method DiffsFormer performs better than Shake-shake by a large margin. 3) Data augmentation can enhance the model stability, as the box of the augmentation is commonly shorter than that of the original. 4) The lower bound of the algorithm increases from the left to the right, suggesting that data augmentation improve the worst-case model performance.

G. Improvements for DiffsFormer

1) *Loss-guided Noise Addition*: Besides the annualized return ratio, information ratio (IR) is another essential measure of the stock forecasting performance. IR is the ratio of the active return of the portfolio divided by the tracking error of its return, where tracking error identifies the level of consistency with which a portfolio “tracks” the performance of an index⁴. In other words, it measures the stability and

⁴https://en.wikipedia.org/wiki/Information_ratio

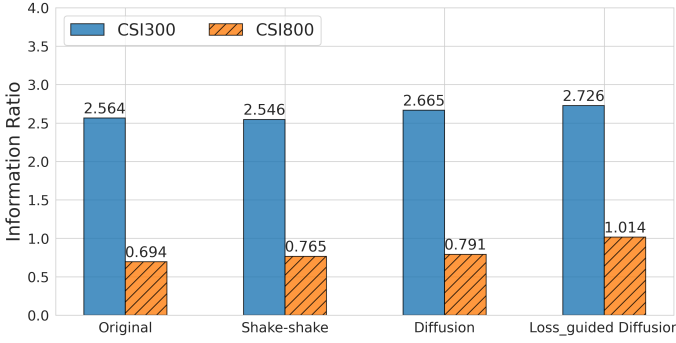


Fig. 12: Information ratio by different training data.

generalization of the model. We discover that there exists some easy-fitted points in the dataset, and we assume that alleviating the overfitting issue of these extreme data points can decrease volatility and lead to a higher information ratio. To answer **RQ3**, we plot the training loss over time in Figure 9. The loss for stock forecasting is quite low during the stock crash that occurred between June 2015 to June 2016, we suspect that the cause is the increased proportion of retail investment whose action pattern is simpler. A model that fits too much to the data around 2015 will fail in a recent time as the market pattern becomes more complex; however, it is sub-optimal to remove these data which exaggerates the data scarcity. Thus, we propose a novel strategy called loss-guided noise addition. Specifically, we leverage training loss as a proxy and add stronger noise to data points with lower training loss. As shown in Figure 9c, the loss-guided diffusion makes the training losses flatter compared to uniform noise-addition, alleviating the overfitting issue and achieving higher information ratio as verified in Figure 12, where IR on CSI300 and CSI800 with different data are reported. We observe that (1) data augmentation can increase the IR of the model; (2) DM outperforms shake-shake; (3) loss-guided diffusion can further increase IR and decrease the volatility.

2) *Time Efficiency Improvement*: From previous analysis in §IV-A, it is obvious to see that there is no need to optimize $\epsilon_\theta(\mathbf{x}_t, t)$ for $t > T'$ under our transfer learning framework. Since DMs are time-consuming, we develop a trick to speed up the training of the framework. Concretely, we initialize α and β with total diffusion steps T to ensure correctness; however, we sample *training step* t from $Uniform\{1, 2, \dots, T'\}$ instead of $Uniform\{1, 2, \dots, T\}$. The loss curves with maximum sampling steps within the set $\{100, 300, 500, 700, 1000\}$ are elucidated in Figure 13. Note that the figure represents the average loss within sampling step 100 instead of diffusion step 1000. We discover that with the decrease of sampling steps, DMs embrace with a more sharp loss curve, which means they can converge faster.

3) *Data Collision*: In modern stock market, several models which utilize the same copy of data coexist. If they provide with the same recommendation, the influx of more money into the market will make some factors ineffective, and models relying on these factors get failed [12]. In Table IX, we

TABLE IX: Performance with different inputs.

	Original only	Union of the two	Augmented only
Performance	0.278941	0.298965	0.312679

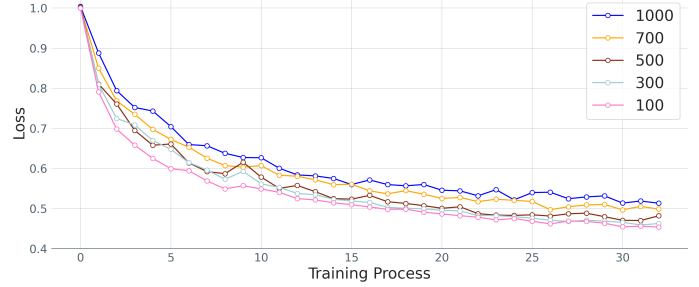


Fig. 13: Loss curves with different sampling steps.

report the performance of model with three types of input: the original data, the augmented data, and the union of these two datasets. We have several observations: 1) Data augmentation is effective since it boosts the model performance; 2) Utilizing both the original data and the augmented data is inferior to using augmented data only. Given the assumption that original data has better data quality than the augmented data, and our test data is real-world, we guess the reason is data collision, which our proposed model can address. Furthermore, in Figure 11, we surprisingly observe that directly tweaking feature (*e.g.*, applying noise) can bring performance improvement, further indicating that data collision is severe in dataset.

VI. CONCLUSION AND FUTURE WORK

In this work, we reveal that one of the most challenges that stock forecasting task faces is data scarcity. To address the issue, we propose DiffsFormer, a novel conditional diffusion Transformer framework for stock forecasting, which focuses on augmenting time-series stock data with help of label and industry information. To distill new knowledge and information, we incorporate transfer learning in DM by training it in a larger source domain and synthesizing with data in target domain. Furthermore, we develop several novel mechanisms to boost the overall model performance, decrease the volatility, increase the time efficiency.

This work takes a first step to do data augmentation in stock forecasting with diffusion. There are some future work directions and limitations can provide valuable insights for further research. We discover that conditionings such as industry sector can improve the performance, hence it is possible to enhance the performance of target stocks by editing factors into a specific industry or generating stocks with specific market value. Additionally, this work highlights the issues of data collision and homogeneity in stock forecasting, further research could involve developing techniques to identify and handle data collisions, as well as strategies to introduce diversity and heterogeneity in the training data explicitly.

REFERENCES

- [1] J. Zou, Q. Zhao, Y. Jiao, H. Cao, Y. Liu, Q. Yan, E. Abbasnejad, L. Liu, and J. Q. Shi, "Stock market prediction via deep learning techniques: A survey," *arXiv preprint arXiv:2212.12717*, 2022.
- [2] L. Zhang, C. C. Aggarwal, and G. Qi, "Stock price prediction via discovering multi-frequency trading patterns," in *KDD*. ACM, 2017, pp. 2141–2149.
- [3] F. Feng, H. Chen, X. He, J. Ding, M. Sun, and T. Chua, "Enhancing stock movement prediction with adversarial training," in *IJCAI*. ijcai.org, 2019, pp. 5843–5849.
- [4] W. Xu, W. Liu, L. Wang, Y. Xia, J. Bian, J. Yin, and T. Liu, "HIST: A graph-based framework for stock trend forecasting via mining concept-oriented shared information," *CoRR*, vol. abs/2110.13716, 2021.
- [5] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever, and M. Chen, "Glide: Towards photorealistic image generation and editing with text-guided diffusion models," *arXiv preprint arXiv:2112.10741*, 2021.
- [6] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," *arXiv preprint arXiv:2204.06125*, 2022.
- [7] Y. Tashiro, J. Song, Y. Song, and S. Ermon, "Csdi: Conditional score-based diffusion models for probabilistic time series imputation," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 804–24 816, 2021.
- [8] N. Chen, Y. Zhang, H. Zen, R. J. Weiss, M. Norouzi, and W. Chan, "Wavegrad: Estimating gradients for waveform generation," *arXiv preprint arXiv:2009.00713*, 2020.
- [9] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.
- [10] W. Peebles and S. Xie, "Scalable diffusion models with transformers," *arXiv preprint arXiv:2212.09748*, 2022.
- [11] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.
- [12] H. Jacobs and S. Müller, "Anomalies across the globe: Once public, no longer existent?" *Journal of Financial Economics*, vol. 135, no. 1, pp. 213–230, 2020.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] J. Chung, Ç. Gülgehr, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014.
- [15] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional LSTM and other neural network architectures," *Neural Networks*, vol. 18, no. 5-6, pp. 602–610, 2005.
- [16] H. Li, Y. Shen, and Y. Zhu, "Stock price prediction using attention-based multi-input LSTM," in *ACML*, ser. Proceedings of Machine Learning Research, vol. 95. PMLR, 2018, pp. 454–469.
- [17] G. Ding and L. Qin, "Study on the prediction of stock price based on the associated network model of LSTM," *Int. J. Mach. Learn. Cybern.*, vol. 11, no. 6, pp. 1307–1317, 2020.
- [18] A. M. Rather, A. Agarwal, and V. N. Sastry, "Recurrent neural network and a hybrid model for prediction of stock returns," *Expert Syst. Appl.*, vol. 42, no. 6, pp. 3234–3241, 2015.
- [19] D. Chen, Y. Zou, K. Harimoto, R. Bao, X. Ren, and X. Sun, "Incorporating fine-grained events in stock movement prediction," *CoRR*, vol. abs/1910.05078, 2019.
- [20] S. Deng, N. Zhang, W. Zhang, J. Chen, J. Z. Pan, and H. Chen, "Knowledge-driven stock trend prediction and explanation via temporal convolutional network," in *WWW (Companion Volume)*. ACM, 2019, pp. 678–685.
- [21] W. Lu, J. Li, J. Wang, and L. Qin, "A cnn-bilstm-am method for stock price prediction," *Neural Comput. Appl.*, vol. 33, no. 10, pp. 4741–4753, 2021.
- [22] S. K. Chandar, "Convolutional neural network for stock trading using technical indicators," *Autom. Softw. Eng.*, vol. 29, no. 1, p. 16, 2022.
- [23] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR (Poster)*, 2018.
- [24] C. Xu, H. Huang, X. Ying, J. Gao, Z. Li, P. Zhang, J. Xiao, J. Zhang, and J. Luo, "HGNN: hierarchical graph neural network for predicting the classification of price-limit-hitting stocks," *Inf. Sci.*, vol. 607, pp. 783–798, 2022.
- [25] W. Li, R. Bao, K. Harimoto, D. Chen, J. Xu, and Q. Su, "Modeling the stock relation with graph network for overnight stock movement prediction," in *IJCAI*. ijcai.org, 2020, pp. 4541–4547.
- [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *NIPS*, 2017, pp. 5998–6008.
- [27] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction," in *IJCAI*. ijcai.org, 2020, pp. 4640–4646.
- [28] J. Yoo, Y. Soun, Y. Park, and U. Kang, "Accurate multivariate stock movement prediction via data-axis transformer with multi-level contexts," in *KDD*. ACM, 2021, pp. 2037–2045.
- [29] L. Yang, T. L. J. Ng, B. Smyth, and R. Dong, "HTML: hierarchical transformer-based multi-task learning for volatility prediction," in *WWW. ACM / IW3C2*, 2020, pp. 441–451.
- [30] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 8857–8868.
- [31] T. Yan, H. Zhang, T. Zhou, Y. Zhan, and Y. Xia, "Scoregrad: Multivariate probabilistic time series forecasting with continuous energy-based generative models," *CoRR*, vol. abs/2106.10121, 2021.
- [32] Y. Li, X. Lu, Y. Wang, and D. Dou, "Generative time series forecasting with diffusion, denoise, and disentanglement," in *NeurIPS*, 2022.
- [33] H. Wen, Y. Lin, Y. Xia, H. Wan, R. Zimmermann, and Y. Liang, "Diffstg: Probabilistic spatio-temporal graph forecasting with denoising diffusion models," *CoRR*, vol. abs/2301.13629, 2023.
- [34] L. Lin, Z. Li, R. Li, X. Li, and J. Gao, "Diffusion models for time series applications: A survey," *CoRR*, vol. abs/2305.00624, 2023.
- [35] W. Wang, Y. Xu, F. Feng, X. Lin, X. He, and T.-S. Chua, "Diffusion recommender model," *arXiv preprint arXiv:2304.04971*, 2023.
- [36] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *arXiv preprint arXiv:2010.02502*, 2020.
- [37] R. He, S. Sun, X. Yu, C. Xue, W. Zhang, P. H. S. Torr, S. Bai, and X. Qi, "Is synthetic data from generative models ready for image recognition?" in *ICLR*. OpenReview.net, 2023.
- [38] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 684–10 695.
- [39] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," *arXiv preprint arXiv:2011.13456*, 2020.
- [40] P. Dhariwal and A. Nichol, "Diffusion models beat gans on image synthesis," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8780–8794, 2021.
- [41] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *MICCAI (3)*, vol. 9351, 2015, pp. 234–241.
- [42] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. C. Courville, "Film: Visual reasoning with a general conditioning layer," in *AAAI*. AAAI Press, 2018, pp. 3942–3951.
- [43] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," in *ICLR*, 2019.
- [44] P. Goyal, P. Dollár, R. B. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch SGD: training imagenet in 1 hour," *CoRR*, vol. abs/1706.02677, 2017.
- [45] X. Yang, W. Liu, D. Zhou, J. Bian, and T. Liu, "Qlib: An ai-oriented quantitative investment platform," *CoRR*, vol. abs/2009.11189, 2020.
- [46] M. Wang, M. Zhang, J. Guo, and W. Jia, "MTMD: multi-scale temporal memory learning and efficient debiasing framework for stock trend forecasting," *CoRR*, vol. abs/2212.08656, 2022.
- [47] H. Lin, D. Zhou, W. Liu, and J. Bian, "Learning multiple stock trading patterns with temporal routing adaptor and optimal transport," in *KDD*. ACM, 2021, pp. 1017–1026.
- [48] Z. Li, D. Yang, L. Zhao, J. Bian, T. Qin, and T. Liu, "Individualized indicator for all: Stock-wise technical indicator optimization with stock embedding," in *KDD*. ACM, 2019, pp. 894–902.
- [49] J. Shipard, A. Wiliem, K. N. Thanh, W. Xiang, and C. Fookes, "Diversity is definitely needed: Improving model-agnostic zero-shot classification via stable diffusion," in *Computer Vision and Pattern Recognition Workshop on Generative Models for Computer Vision*, 2023.
- [50] X. Gastaldi, "Shake-shake regularization," *CoRR*, vol. abs/1705.07485, 2017.